Lie-Invariant Dual Quaternion MPCC for Quadrotors

Bryan S. Guevara, Luis F. Simancas, and Tiago Nascimento

***Abstract*—This paper presents a Model Predictive Contouring Control (MPCC) framework for quadrotor UAVs in which the path-following error is formulated directly on the Lie group $SE(3)$ using unit dual quaternions. Unlike conventional MPCC approaches that decompose position and orientation errors independently in Euclidean space, the proposed method computes lag and contouring error components through the logarithmic map of the dual quaternion error, yielding a geometrically consistent, singularity-free decomposition in the Lie algebra $\mathfrak{se}(3)$. The translational error is corrected via the left Jacobian of $SO(3)$ to account for the rotation-translation coupling intrinsic to $\mathfrak{se}(3)$, and projected onto the trajectory tangent to separate lag and contouring terms under the Ad-invariant metric. The arc-length progress is embedded as a system state and augmented with second-order virtual progress dynamics, enabling the solver to jointly optimize the physical control inputs and the progress acceleration within a single nonlinear program. This preserves a torque-driven quadrotor model while avoiding unrealistically abrupt changes in the virtual traversal speed. The resulting OCP is solved in real time via Sequential Quadratic Programming with Real-Time Iteration (SQP-RTI) using the ACADOS framework. In Model-in-the-Loop simulations over an $80\,\mathrm{m}$ 3D Lissajous trajectory, the proposed DQ-MPCC consistently reduces contouring and orientation error relative to a Euclidean MPCC baseline, while exposing a clear trade-off in lag error that is analyzed explicitly. These results support the proposed formulation as a geometrically consistent alternative when rigid-body pose accuracy is prioritized together with real-time feasibility.**

***Index Terms*—Model Predictive Contouring Control, Lie Groups, Dual Quaternions, Quadrotor UAV, $SE(3)$, Nonlinear MPC, Arc-Length Parametrization.**

I. Introduction

HIGH-SPEED autonomous flight requires controllers that simultaneously maximize traversal speed and maintain precise spatial accuracy along a prescribed path, a regime that is well known to demand large attitude excursions and strongly coupled translational–rotational motion in quadrotor systems [16], [17]. Model Predictive Contouring Control (MPCC) [1] addresses this challenge by decoupling *spatial* tracking accuracy from *temporal* progress: rather than enforcing a fixed time schedule, the controller treats the arc-length progress as an additional optimization variable, enabling autonomous speed modulation—accelerating on straight segments and decelerating in high-curvature regions. Since its introduction in autonomous racing [1], MPCC has been extended to quadrotor flight [2], ground vehicles [3], and multi-agent systems, demonstrating consistent advantages over time-parametrized MPC in agile, constraint-aware motion.

Despite these advances, virtually all existing MPCC formulations decompose the path-following error in Euclidean space $\mathbb{R}^3$, relying on Euler angles or rotation matrices to handle the orientational degrees of freedom [2], [3]. These representations carry well-documented drawbacks. Euler angles suffer from gimbal lock and exhibit discontinuities at $\pm\pi$ that corrupt gradient information inside the nonlinear program. Rotation matrices require six orthonormality constraints, inflating the problem size and complicating sensitivity analysis. More fundamentally, treating position and orientation as independent quantities discards the coupled rigid-body structure of $SE(3)$: any error metric defined separately on $\mathbb{R}^3 \times SO(3)$ fails to capture the interaction between rotational and translational motion that is intrinsic to the Special Euclidean group.

Unit dual quaternions offer a principled remedy. A unit dual quaternion $\hat{\boldsymbol{q}} \in \mathbb{H}_d$ encodes a full $SE(3)$ pose in a unified 8-dimensional algebra subject to only two scalar constraints [4], [6], compared to twelve for homogeneous matrices. Beyond compactness, dual quaternions inherit the Lie group structure of $SE(3)$: the exponential and logarithmic maps are closed-form, the error between two poses is a well-defined element of the Lie algebra $\mathfrak{se}(3)$, and the associated Ad-invariant metric provides a geometrically consistent notion of distance on the manifold. Control designs based on these properties have demonstrated singularity-free tracking of coupled rotational-translational motion [4], [5], [7], yet no prior work has exploited them within an MPCC framework for path-following speed optimization.

This paper closes that gap. We present a Lie-invariant MPCC framework for quadrotor UAVs in which the lag and contouring error components are computed directly in $\mathfrak{se}(3)$ via the logarithmic map of the dual quaternion pose error. The arc-length progress is embedded as a system state and augmented with second-order virtual-progress dynamics $\dot{s} = v_s,\ \dot{v}_s = a_s$, allowing the solver to jointly optimize physical control inputs and the progress acceleration within a single nonlinear program. This choice is inspired by the progress regularisation strategy in [2], but adapted here to a torque-driven MPCC formulation so that the baseline actuation structure is preserved. The resulting optimal control problem is solved in real time using Sequential Quadratic Programming with Real-Time Iteration (SQP-RTI) inside the ACADOS

Manuscript received January 14, 2025; revised March 3, 2025.

Bryan S. Guevara is with the Laboratório de Automação e Sistemas de Robótica (LaSER), Universidade Federal da Paraíba (UFPB), João Pessoa, PB 58051-900, Brazil. bguevara@laser.ufpb.br

Luis F. Simancas is with Instituto de Automática (INAUT), Universidad Nacional de San Juan–CONICET, Argentina. lsimancas@inaut.unsj.edu.ar

Tiago Nascimento is with the Laboratório de Automação e Sistemas de Robótica (LaSER), Universidade Federal da Paraíba (UFPB), João Pessoa, PB 58051-900, Brazil. tiago@laser.ufpb.br

This work has been supported by the Ministry of Science and Innovation of the Spanish Government through the research project PID2021-122685OBI00 and the grant PRE2019-088069 for the training of research PhD staff.

0000–0000/00$00.00 © 2025 IEEE

framework [12].

The main contributions of this work are as follows:

1) **Dual-quaternion MPCC on $SE(3)$:** To the best of our knowledge, we present the first MPCC formulation for quadrotors that operates directly on the Lie group $SE(3)$ using unit dual quaternions, extending contouring control from Euclidean path following to full rigid-body motion with coupled rotation and translation.
2) **Lie-algebra lag-contouring decomposition:** We introduce a geometrically consistent decomposition of the path-following error in the Lie algebra $\mathfrak{se}(3)$. The pose error is obtained from the dual quaternion logarithmic map; its translational component is corrected via the left Jacobian of $SO(3)$ and projected onto the trajectory tangent to yield coordinate-invariant lag and contouring terms consistent with the Ad-invariant metric of $\mathfrak{se}(3)$.
3) **Second-order virtual-progress augmentation:** We augment the MPCC progress variable with explicit second-order dynamics, introducing a bounded progress acceleration as the virtual optimization input. In contrast to actuator-level formulations such as [2], this preserves the thrust–torque structure of the quadrotor model used in both the Euclidean baseline and the proposed DQ-MPCC, while still enforcing smooth and dynamically plausible progress evolution.
4) **Real-time validation:** We demonstrate real-time feasibility at $100\,\mathrm{Hz}$ on an $80\,\mathrm{m}$ 3D Lissajous path using SQP-RTI, and show through MiL experiments that the proposed formulation improves contouring and orientation accuracy while preserving comparable traversal times with respect to the Euclidean baseline.

The remainder of the paper is organized as follows. Section II reviews dual quaternion algebra and the relevant Lie group structure. Section III derives the UAV dynamics in dual quaternion form. Section IV presents the Lie-invariant MPCC formulation. Section VI details the implementation and solver configuration. Section VII reports simulation results, and Section **??** concludes with directions for future work.

DQ-MPCC.png

Fig. 1: Geometric interpretation of the Lie-invariant lag–contouring decomposition on $SE(3)$. The desired trajectory $\boldsymbol{\gamma}(s)$ defines a curve on $SE(3)$, parameterized by the arc length s. At time step k, the reference and current poses are denoted by q_k^d and q_k, respectively, with relative error $q^e = q_k^{d*} \otimes q_k \in SE(3)$. Applying the logarithmic map yields the Lie-algebra error $\boldsymbol{\xi}^e = (\boldsymbol{\phi}^e, \boldsymbol{\rho}) \in \mathfrak{se}(3)$ in the tangent space at q_k^d. Here, $\boldsymbol{\phi}^e \in \mathfrak{so}(3)$ is the rotational error, whereas $\boldsymbol{\rho} \in \mathbb{R}^3$ is the translational component of the logarithmic pose error, and thus is not the Euclidean position difference, since it is coupled to the relative orientation. Using the unit path tangent $\boldsymbol{e}_t^d(s) = d\boldsymbol{p}^d/ds$, $\boldsymbol{\rho}$ is decomposed into a lag component $\boldsymbol{\rho}_{\mathrm{lag}}$, aligned with the path, and a contouring component $\boldsymbol{\rho}_{\mathrm{cont}}$, orthogonal to it. This yields a geometrically consistent separation of longitudinal and lateral path-following errors in $\mathfrak{se}(3)$.

II. Mathematical Preliminaries

A. Dual Quaternion Algebra

A dual number is an expression $a + \varepsilon b$, where $a, b \in \mathbb{R}$ and the dual unit ε satisfies $\varepsilon^2 = 0$, $\varepsilon \neq 0$. Extending this to the quaternion algebra $\mathbb{H}$, a *dual quaternion* is

$$\hat{\boldsymbol{q}} = q_r + \varepsilon\, q_d, \qquad q_r,\, q_d \in \mathbb{H}, \tag{1}$$

where q_r is the *real part* and q_d is the *dual part*. This representation is standard in rigid-body kinematics and provides a compact pose parametrization that remains closed under composition and inversion [8], [9]. The product of two dual quaternions follows from the Hamilton product $\otimes$ applied to both parts:

$$\hat{\boldsymbol{q}}_1 \otimes \hat{\boldsymbol{q}}_2 = q_{r_1} \otimes q_{r_2} + \varepsilon(q_{r_1} \otimes q_{d_2} + q_{d_1} \otimes q_{r_2})\,. \tag{2}$$

A dual quaternion represents a rigid-body pose in $SE(3)$ when it satisfies the *unit constraints*

$$\|q_r\| = 1, \qquad \langle q_r,\, q_d \rangle = 0, \tag{3}$$

where $\langle \cdot, \cdot \rangle$ denotes the scalar (dot) product of two quaternions treated as vectors in $\mathbb{R}^4$. These conditions characterize the unit dual quaternions associated with rigid motions and ensure consistency with the geometry of $SE(3)$ [9]. Given a rotation parametrized by angle θ about unit axis $\hat{\boldsymbol{n}} \in \mathbb{R}^3$ and a translation $\boldsymbol{t} \in \mathbb{R}^3$, the corresponding unit dual quaternion is

$$\hat{\boldsymbol{q}} = q_r + \frac{\varepsilon}{2}\, q_t \otimes q_r, \qquad q_r = \begin{bmatrix} \cos\frac{\theta}{2} \\ \hat{\boldsymbol{n}} \sin\frac{\theta}{2} \end{bmatrix}, \quad q_t = \begin{bmatrix} 0 \\ \boldsymbol{t} \end{bmatrix}. \tag{4}$$

The conjugate of a unit dual quaternion is $\hat{\boldsymbol{q}}^* = q_r^* + \varepsilon\, q_d^*$, and the inverse equals the conjugate, $\hat{\boldsymbol{q}}^{-1} = \hat{\boldsymbol{q}}^*$, by virtue of the unit constraint (3). The translation and rotation are recovered from any unit dual quaternion as

$$\boldsymbol{t} = 2\left(q_d \otimes q_r^*\right)\big|_{\mathrm{vec}}, \qquad R = R(q_r) \in SO(3), \tag{5}$$

where $(\cdot)|_{\mathrm{vec}}$ denotes the vector part of a quaternion.

B. Lie Group Structure and Logarithmic Map

The set of unit dual quaternions satisfying (3) forms the group $\mathrm{Spin}(3) \ltimes \mathbb{R}^3$, the double cover of $SE(3)$: every rigid-body pose corresponds to exactly two antipodal unit dual quaternions $\pm\hat{\boldsymbol{q}}$ [6], [9]. Its Lie algebra $\mathfrak{se}(3)$ consists of six-dimensional vectors

$$\boldsymbol{\xi} = \begin{pmatrix} \boldsymbol{\phi} \\ \boldsymbol{\rho} \end{pmatrix} \in \mathbb{R}^6, \tag{6}$$

where $\boldsymbol{\phi} \in \mathbb{R}^3$ is the rotation vector and $\boldsymbol{\rho} \in \mathbb{R}^3$ is the coupled translational component.

The **logarithmic map** $\mathrm{Log} : \mathbb{H}_d \to \mathfrak{se}(3)$ proceeds in two steps. First, the rotation angle and axis are extracted from the real part q_r:

$$\theta = 2\,\mathrm{atan2}\big(\|q_{r,\mathrm{vec}}\|,\ q_{r,0}\big), \qquad \hat{\boldsymbol{n}} = \frac{q_{r,\mathrm{vec}}}{\|q_{r,\mathrm{vec}}\|}, \tag{7}$$

and the translation $\boldsymbol{t}$ is recovered via (5). Then the Lie algebra element is

$$\mathrm{Log}(\hat{\boldsymbol{q}}) = \begin{pmatrix} \boldsymbol{\phi} \\ \boldsymbol{\rho} \end{pmatrix} = \begin{pmatrix} \theta\,\hat{\boldsymbol{n}} \\ \boldsymbol{t} \end{pmatrix}. \tag{8}$$

Throughout the paper, quantities associated with the desired (reference) pose carry a superscript d: $\hat{\boldsymbol{q}}^d$, q_r^d, $\boldsymbol{p}^d$, R^d, etc. Hemisphere consistency is enforced by

$$q_r \ \leftarrow\ \mathrm{sign}(q_{r,0})\, q_r \tag{9}$$

before evaluating (7)–(8). These logarithmic coordinates follow the standard Lie-theoretic local representation used in robotics, where Jacobian corrections map finite group displacements to tangent perturbations [10], [11].

III. Dual Quaternion Dynamics of the Quadrotor

A. Classical Quadrotor Model

Consider a quadrotor of mass m and inertia matrix $\mathbf{J} \in \mathbb{R}^{3\times 3}$. Let $\boldsymbol{p} \in \mathbb{R}^3$ denote the position in the inertial frame, $q_r \in \mathbb{S}^3$ the unit attitude quaternion, $\boldsymbol{v} \in \mathbb{R}^3$ the translational velocity in the inertial frame, and $\boldsymbol{\omega} \in \mathbb{R}^3$ the angular velocity in the body frame. The equations of motion are:

$$\dot{\boldsymbol{p}} = \boldsymbol{v}, \tag{10}$$

$$\dot{q}_r = \tfrac{1}{2}\, q_r \otimes \boldsymbol{\omega}, \tag{11}$$

$$\dot{\boldsymbol{v}} = \frac{f}{m}\, R(q_r)\, \boldsymbol{e}_3 + \boldsymbol{g}, \tag{12}$$

$$\dot{\boldsymbol{\omega}} = \mathbf{J}^{-1}\big(\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})\big), \tag{13}$$

where $f \in \mathbb{R}$ is the total thrust, $\boldsymbol{\tau} \in \mathbb{R}^3$ is the control torque vector, $\boldsymbol{e}_3 = [0, 0, 1]^\top$ is the body-frame thrust direction, and $\boldsymbol{g} = [0, 0, -g]^\top$ m/s^2 is the gravitational acceleration in the inertial frame.

B. Dual Quaternion State Representation

The separate kinematics (10)–(11) are unified into a single pose equation by assembling the unit dual quaternion (4):

$$\hat{\boldsymbol{q}} = q_r + \varepsilon\, q_d, \tag{14}$$

where $q_r \in \mathbb{R}^4$ is the real part encoding the attitude and $q_d \in \mathbb{R}^4$ is the dual part encoding the translation coupled with the rotation, defined as

$$q_d \ \triangleq\ \tfrac{1}{2}\, q_t \otimes q_r, \qquad \text{with} \qquad q_t = [0,\ \boldsymbol{p}^\top]^\top. \tag{15}$$

The angular and linear velocities are packed into the *dual twist*, a pure dual quaternion

$$\hat{\boldsymbol{\omega}} = \boldsymbol{\omega} + \varepsilon\, \boldsymbol{v}_b, \tag{16}$$

where

$$\boldsymbol{v}_b \ \triangleq\ q_r^* \otimes \boldsymbol{v} \otimes q_r \ =\ \mathrm{Ad}_{q_r^*}(\boldsymbol{v}) \tag{17}$$

is the translational velocity rotated from the inertial frame into the body frame via the adjoint action. Because $\hat{\boldsymbol{\omega}}$ is pure (both scalar parts vanish), it carries exactly six independent degrees of freedom collected in $\boldsymbol{\xi} = [\boldsymbol{\omega}^\top,\ \boldsymbol{v}_b^\top]^\top \in \mathbb{R}^6$.

With these definitions the pose kinematics reduce to

$$\dot{\hat{\boldsymbol{q}}} = \tfrac{1}{2}\, \hat{\boldsymbol{q}} \otimes \hat{\boldsymbol{\omega}}, \tag{18}$$

and the dual twist dynamics decompose as

$$\dot{\hat{\boldsymbol{\omega}}} = \hat{\boldsymbol{f}} + \hat{\boldsymbol{u}}, \tag{19}$$

where the drift and control dual quaternions are, respectively,

$$\hat{\boldsymbol{f}} = -\mathbf{J}^{-1}\big(\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})\big) + \varepsilon\big(\boldsymbol{v}_b \times \boldsymbol{\omega} + \mathrm{Ad}_{q_r^*}(\boldsymbol{g})\big)\,, \tag{20}$$

$$\hat{\boldsymbol{u}} = \mathbf{J}^{-1}\boldsymbol{\tau} + \varepsilon\left(\frac{f}{m}\,\boldsymbol{e}_3\right). \tag{21}$$

C. Compact State Equation

Collecting the pose $\hat{\boldsymbol{q}} = q_r + \varepsilon\, q_d$ and the dual twist $\hat{\boldsymbol{\omega}} = \boldsymbol{\omega} + \varepsilon\, \boldsymbol{v}_b$ into a single state, the complete UAV dynamics in dual quaternion form are:

$$\boxed{\frac{d}{dt}\begin{bmatrix} \hat{\boldsymbol{q}} \\ \hat{\boldsymbol{\omega}} \end{bmatrix} = \begin{bmatrix} \tfrac{1}{2}\,\hat{\boldsymbol{q}} \otimes \hat{\boldsymbol{\omega}} \\ \hat{\boldsymbol{f}} + \hat{\boldsymbol{u}} \end{bmatrix}.} \tag{22}$$

This compact form follows the usual Lie-group treatment of rigid-body kinematics and dynamics, where pose evolution is written directly on the motion group instead of splitting translation and attitude into unrelated coordinate charts [13], [14].

For numerical implementation, $\hat{\boldsymbol{q}} = q_r + \varepsilon\, q_d$ is stored as its eight scalar components and $\hat{\boldsymbol{\omega}} = \boldsymbol{\omega} + \varepsilon\, \boldsymbol{v}_b$ as its six vector components $\boldsymbol{\xi}$, yielding the state vector

$$\boldsymbol{x} = \big[\underbrace{q_r \in \mathbb{R}^4}_{\text{attitude}},\ \underbrace{q_d \in \mathbb{R}^4}_{\text{dual part}},\ \underbrace{\boldsymbol{\omega} \in \mathbb{R}^3}_{\text{ang. vel.}},\ \underbrace{\boldsymbol{v}_b \in \mathbb{R}^3}_{\text{body lin. vel.}}\big]^\top \in \mathbb{R}^{14}, \tag{23}$$

and the control input $\boldsymbol{u} = [f,\ \tau_1,\ \tau_2,\ \tau_3]^\top \in \mathbb{R}^4$. The unit-norm constraint $\|q_r\| = 1$ is maintained via a Baumgarte-type correction $K(1 - \|q_r\|)\, q_r$ with $K = 10$ added to $\dot{q}_r$ inside the integrator, supplemented by a soft penalty constraint in the OCP formulation. This is a classical constraint-stabilization device used here only to keep the numerical integration close to the unit-quaternion manifold [19].

D. Reference Trajectory and Arc-Length Parametrization

MPCC requires a *spatial* reference indexed by distance along the path rather than reference path by time $\boldsymbol{p}^d(t)$, so that the optimizer can freely choose how fast to traverse each segment. The arc length is

$$s(t) = \int_0^t \left\|\dot{\boldsymbol{p}}^d(\tau)\right\| d\tau, \tag{24}$$

and the inverse $t(s)$ is recovered by cubic spline interpolation. The reference pose at station s is

$$\boldsymbol{\gamma}(s) = \left(\boldsymbol{p}^d(s),\ q_r^d(s)\right) \in SE(3), \tag{25}$$

where the desired attitude $q_r^d(s)$ is constructed from the path geometry. First, the unit tangent

$$\boldsymbol{t}^d(s) = \frac{d\boldsymbol{p}^d/ds}{\|d\boldsymbol{p}^d/ds\|} \tag{26}$$

defines the desired heading direction. Next, the path curvature vector is approximated by

$$\boldsymbol{\kappa}^d(s) = \frac{d\boldsymbol{t}^d}{ds}, \tag{27}$$

and combined with a nominal reference speed $\bar{v}_s$ to form the feedforward lateral acceleration

$$\boldsymbol{a}_{\text{lat}}^d(s) = \bar{v}_s^2\, \boldsymbol{\kappa}^d(s). \tag{28}$$

The desired thrust direction is then chosen as

$$\boldsymbol{b}_3^d(s) = \frac{\boldsymbol{a}_{\text{lat}}^d(s) + g\,\boldsymbol{e}_3}{\|\boldsymbol{a}_{\text{lat}}^d(s) + g\,\boldsymbol{e}_3\|}, \tag{29}$$

which is clipped to a maximum admissible tilt angle in order to keep the reference physically realizable. The desired body x-axis is the projection of the tangent onto the plane orthogonal to $\boldsymbol{b}_3^d$,

$$\boldsymbol{b}_1^d(s) = \frac{\left(\boldsymbol{I}_3 - \boldsymbol{b}_3^d \boldsymbol{b}_3^{d\top}\right) \boldsymbol{t}^d(s)}{\left\|\left(\boldsymbol{I}_3 - \boldsymbol{b}_3^d \boldsymbol{b}_3^{d\top}\right) \boldsymbol{t}^d(s)\right\|}, \tag{30}$$

with $\boldsymbol{b}_2^d(s) = \boldsymbol{b}_3^d(s) \times \boldsymbol{b}_1^d(s)$. The reference rotation matrix is finally assembled as

$$R^d(s) = \begin{bmatrix}\boldsymbol{b}_1^d(s) & \boldsymbol{b}_2^d(s) & \boldsymbol{b}_3^d(s)\end{bmatrix} \in SO(3), \tag{31}$$

and mapped to the quaternion $q_r^d(s)$. Unlike a yaw-only reference, this construction makes the orientation reference compatible with the centripetal acceleration required for fast path following.

IV. Lie-Invariant MPCC Formulation

Classical MPCC decomposes the path-following error in Euclidean space using the Frenet–Serret frame [1], thereby treating position and orientation separately and disregarding the underlying manifold structure of $SE(3)$. In contrast, the proposed formulation performs the decomposition directly in $\mathfrak{se}(3)$ through the logarithmic map of the dual-quaternion error, together with the left-Jacobian correction introduced in Section **??**.

A. Baseline MPCC

To assess the benefit of the proposed geometric formulation, the DQ-MPCC is benchmarked against a standard Euclidean MPCC baseline. Given the reference position $\boldsymbol{p}^d(s)$ and its unit tangent $\boldsymbol{t}(s)$, the position tracking error $\boldsymbol{e}_t = \boldsymbol{p}^d(s) - \boldsymbol{p}$ is decomposed into lag and contouring components as

$$\boldsymbol{e}_\ell = \left(\boldsymbol{t}^\top \boldsymbol{e}_t\right) \boldsymbol{t}, \qquad \boldsymbol{e}_c = \left(\boldsymbol{I}_3 - \boldsymbol{t}\,\boldsymbol{t}^\top\right) \boldsymbol{e}_t, \tag{32}$$

while the attitude error is defined through the quaternion logarithm

$$\boldsymbol{\eta} = \operatorname{Log}(q^{-1} \otimes q^d) \in \mathfrak{so}(3), \tag{33}$$

using the same curvature-compatible attitude reference $q^d(s) = q_r^d(s)$ introduced in (29)–(31). This avoids penalizing a hover-like orientation while the position loop demands the tilt required for aggressive path following.

Both controllers share the same solver configuration, prediction horizon, stage-cost structure (45), and physical constraints (47f)–(47h). Hence, the only difference lies in the pose-error representation: the baseline uses a Euclidean decomposition in $\mathbb{R}^3$ with a separate quaternion attitude error, whereas the proposed DQ-MPCC employs the geometrically consistent logarithmic map on $SE(3)$, as detailed in Section IV. In the baseline cost, $\boldsymbol{\eta}$ replaces $\boldsymbol{\phi}^e$ with the same quadratic penalty structure as in (45). Therefore, any observed performance difference can be attributed solely to the adopted error representation.

B. DQ-MPCC Formulation

Let $\hat{\boldsymbol{q}}^d(s) = \boldsymbol{\gamma}(s)$ denote the reference dual quaternion, evaluated at the current arc-length station s — the fifteenth component of the augmented state (43); the sixteenth component stores the virtual progress speed v_s. The pose error is defined as the left-invariant error on $SE(3)$:

$$\hat{\boldsymbol{q}}^e = \hat{\boldsymbol{q}}^{d\,*}(s) \otimes \hat{\boldsymbol{q}}, \tag{34}$$

which expresses the deviation of the current pose $\hat{\boldsymbol{q}}$ from the reference $\hat{\boldsymbol{q}}^d(s)$ in the body frame of the reference [15]. After applying hemisphere correction (9), the logarithmic map (8) yields the Lie algebra error

$$\boldsymbol{\xi}^e = \operatorname{Log}(\hat{\boldsymbol{q}}^e) = \begin{pmatrix}\boldsymbol{\phi}^e \\ \boldsymbol{\rho}^e\end{pmatrix} \in \mathfrak{se}(3), \tag{35}$$

where $\boldsymbol{\phi}^e \in \mathbb{R}^3$ is the rotational error vector and $\boldsymbol{\rho}^e \in \mathbb{R}^3$ is the raw translational component. As detailed in Section **??**, $\boldsymbol{\rho}^e$ encodes translation in a frame twisted by $\boldsymbol{\phi}^e$ through the screw structure of $\mathfrak{se}(3)$; the corrected translational error $\boldsymbol{\rho}$ is defined in (38).

A key quantity linking Lie-algebra errors to Euclidean position errors is the *left Jacobian* of $SO(3)$, $\boldsymbol{J}_l(\boldsymbol{\phi}) \in \mathbb{R}^{3\times 3}$ [10]:

$$\boldsymbol{J}_l(\boldsymbol{\phi}) = \boldsymbol{I}_3 + \frac{1-\cos\theta}{\theta^2}\,[\boldsymbol{\phi}]_\times + \frac{\theta - \sin\theta}{\theta^3}\,[\boldsymbol{\phi}]_\times^2, \tag{36}$$

where $\theta = \|\boldsymbol{\phi}\|$ and $[\boldsymbol{\phi}]_\times$ is the 3×3 skew-symmetric matrix associated with $\boldsymbol{\phi}$. Its inverse admits the closed-form expression [10]:

$$\boldsymbol{J}_l(\boldsymbol{\phi})^{-1} = \frac{\theta}{2}\cot\frac{\theta}{2}\,\boldsymbol{I}_3 \;-\; \frac{\theta}{2}\,[\hat{\boldsymbol{n}}]_\times \;+\; \left(1 - \frac{\theta}{2}\cot\frac{\theta}{2}\right)\hat{\boldsymbol{n}}\,\hat{\boldsymbol{n}}^\top, \tag{37}$$

where $\hat{\boldsymbol{n}} = \boldsymbol{\phi}/\theta$ is the unit rotation axis. Using the analytical form (37) instead of a numerical matrix inversion avoids the heavier expression graph that `ca.inv` would introduce in the CasADi symbolic framework, yielding cleaner automatic differentiation and faster code generation within the acados solver. For $\theta < 10^{-7}$ both expressions reduce to $\boldsymbol{J}_l \to \boldsymbol{I}_3$ and $\boldsymbol{J}_l^{-1} \to \boldsymbol{I}_3$ to avoid numerical singularity.

To understand why this Jacobian is needed, recall that the translational component $\boldsymbol{\rho}^e$ returned by $\mathrm{Log}(\hat{\boldsymbol{q}}^e)$ via (8) is *not* the Euclidean displacement $\boldsymbol{p}-\boldsymbol{p}^d$. Rather, it is the translation expressed in a frame that has been *twisted* by the rotation $\boldsymbol{\phi}^e$ through the $\mathfrak{se}(3)$ screw structure [10]. The corrected translational error is therefore defined as

$$\boldsymbol{\rho} = \boldsymbol{J}_l(\boldsymbol{\phi}^e)^{-1}\, R^{d\,\top}\big(\boldsymbol{p}-\boldsymbol{p}^d\big), \tag{38}$$

where $R^d = R(q_r^d) \in SO(3)$ is the desired rotation matrix and $\boldsymbol{p}-\boldsymbol{p}^d \in \mathbb{R}^3$ is the position error in the inertial frame. Equation (38) thus defines a vector in $\mathbb{R}^3$ that (i) is expressed in the desired body frame via $R^{d\,\top}$, and (ii) accounts for the rotation-translation coupling of $\mathfrak{se}(3)$ via $\boldsymbol{J}_l^{-1}$.

The corrected translational error $\boldsymbol{\rho}$ (38) is decomposed into components parallel and perpendicular to the instantaneous tangent direction of the reference trajectory. At arc-length station s, this tangent is

$$\boldsymbol{e}_t^d(s) = \frac{\boldsymbol{p}^{d\prime}(s)}{\|\boldsymbol{p}^{d\prime}(s)\|}, \qquad \text{with} \qquad \boldsymbol{p}^{d\prime}(s) = \frac{d\boldsymbol{p}^d}{ds}, \tag{39}$$

where $\boldsymbol{p}^{d\prime}(s)$ is the derivative of the reference position with respect to arc length, available analytically from the CasADi interpolant of Section III-D. Because the parametrization is by arc length, $\|\boldsymbol{p}^{d\prime}\| = 1$ holds exactly along smooth segments, so (39) is well-defined everywhere except at cusps. The lag and contouring components are then

$$\boldsymbol{\rho}_{\mathrm{lag}} = \big(\boldsymbol{e}_t^d\, \boldsymbol{e}_t^{d\top}\big)\boldsymbol{\rho}, \tag{40a}$$

$$\boldsymbol{\rho}_{\mathrm{cont}} = \big(\boldsymbol{I}_3 - \boldsymbol{e}_t^d\, \boldsymbol{e}_t^{d\top}\big)\boldsymbol{\rho}. \tag{40b}$$

The *lag error* $\boldsymbol{\rho}_{\mathrm{lag}}$ measures how far the vehicle lags behind or leads the virtual reference point along the path; the *contouring error* $\boldsymbol{\rho}_{\mathrm{cont}}$ measures lateral deviation from it.

This decomposition is geometrically consistent with the $SE(3)$ structure. Because $\boldsymbol{\rho}$ is obtained via the left Jacobian inverse (38), it resides in the translational subspace of $\mathfrak{se}(3)$ with rotation-translation coupling accounted for. Both $\boldsymbol{\rho}_{\mathrm{lag}}$ and $\boldsymbol{\rho}_{\mathrm{cont}}$ are orthogonal projections under the left-invariant metric

$$\langle \boldsymbol{\xi}_1, \boldsymbol{\xi}_2 \rangle_{\mathfrak{se}(3)} = \lambda\, \boldsymbol{\phi}_1^\top \boldsymbol{\phi}_2 + \mu\, \boldsymbol{\rho}_1^\top \boldsymbol{\rho}_2, \qquad \lambda, \mu > 0, \tag{41}$$

which defines the weighted inner product adopted in this work on the tangent coordinates of $\mathfrak{se}(3)$ [13]. A purely Euclidean decomposition of $\boldsymbol{p}-\boldsymbol{p}^d$ would not respect this structure and would produce biased error signals whenever $\boldsymbol{\phi}^e \neq \boldsymbol{0}$.

C. Optimal Control Problem

To embed arc-length progress as a degree of freedom while preventing unrealistically abrupt changes in the virtual traversal speed, the dynamics (22) are augmented with the progress state $s \in [0, s_{\max}]$, the virtual progress speed v_s, and the virtual progress acceleration input a_s:

$$\dot{s} = v_s, \qquad \dot{v}_s = a_s, \tag{42}$$

yielding the extended state vector

$$\boldsymbol{x}_{\mathrm{a}} = \big[\, \underbrace{\boldsymbol{x} \in \mathbb{R}^{14}}_{\text{UAV state}},\ \underbrace{s \in \mathbb{R}}_{\text{arc length}},\ \underbrace{v_s \in \mathbb{R}}_{\text{virtual speed}} \,\big]^\top \in \mathbb{R}^{16}, \tag{43}$$

and the augmented control vector

$$\boldsymbol{u}_{\mathrm{a}} = \big[\, \underbrace{\boldsymbol{u}^\top \in \mathbb{R}^4}_{\text{physical inputs}},\ \underbrace{a_s \in \mathbb{R}}_{\text{virtual acceleration}} \,\big]^\top \in \mathbb{R}^5, \tag{44}$$

The stage cost at each prediction node is:

$$\begin{aligned}\ell(\boldsymbol{x}_{\mathrm{a}}, \boldsymbol{u}_{\mathrm{a}}) = {} & \boldsymbol{\xi}^{e\top} \boldsymbol{Q}_\phi\, \boldsymbol{\xi}^e + \boldsymbol{\rho}_{\mathrm{lag}}^\top \boldsymbol{Q}_\ell\, \boldsymbol{\rho}_{\mathrm{lag}} + \boldsymbol{\rho}_{\mathrm{cont}}^\top \boldsymbol{Q}_c\, \boldsymbol{\rho}_{\mathrm{cont}} \\ & + \boldsymbol{u}^\top \boldsymbol{W}_u\, \boldsymbol{u} + \boldsymbol{\omega}^\top \boldsymbol{Q}_\omega\, \boldsymbol{\omega} + Q_a\, a_s^2 - Q_s\, v_s.\end{aligned} \tag{45}$$

To avoid double-penalising translation, $\boldsymbol{Q}_\phi \in \mathbb{R}^{6\times 6}$ is structured with upper block $\boldsymbol{Q}_q$ acting on the rotational component $\boldsymbol{\phi}^e$ of $\boldsymbol{\xi}^e$ and a zero lower block, since lateral and longitudinal errors are handled exclusively by $\boldsymbol{Q}_c$ and $\boldsymbol{Q}_\ell$. All tracking and regularisation weights $\boldsymbol{Q}_\ell$, $\boldsymbol{Q}_c$, $\boldsymbol{Q}_q$, $\boldsymbol{Q}_\omega \in \mathbb{R}^{3\times 3}$ are diagonal, allowing independent penalisation per axis and accommodating the typical asymmetry between vertical and horizontal dynamics of a quadrotor. The control effort weight $\boldsymbol{W}_u \in \mathbb{R}^{4\times 4}$ penalises thrust and torques independently, and $Q_s > 0$ is a scalar speed incentive weight. Numerical values of all weights are reported in Table **??**. The scalar $Q_a > 0$ regularises the virtual progress acceleration and is kept fixed in the implementation to isolate the effect of the main MPCC tracking weights during bilevel tuning.

The terminal cost retains all error terms but omits control regularization:

$$\ell_e(\boldsymbol{x}_{\mathrm{a}}) = \boldsymbol{\xi}^{e\top} \boldsymbol{Q}_\phi\, \boldsymbol{\xi}^e + \boldsymbol{\rho}_{\mathrm{lag}}^\top \boldsymbol{Q}_\ell\, \boldsymbol{\rho}_{\mathrm{lag}} + \boldsymbol{\rho}_{\mathrm{cont}}^\top \boldsymbol{Q}_c\, \boldsymbol{\rho}_{\mathrm{cont}} + \boldsymbol{\omega}^\top \boldsymbol{Q}_\omega\, \boldsymbol{\omega}. \tag{46}$$

The complete OCP of DQ-MPCC over a prediction horizon of N steps is:

$$\underset{\boldsymbol{x}_{\mathrm{a},0:N},\, \boldsymbol{u}_{\mathrm{a},0:N-1}}{\text{minimize}} \quad \sum_{k=0}^{N-1} \ell(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k}) + \ell_e(\boldsymbol{x}_{\mathrm{a},N}) \tag{47a}$$

$$\text{subject to} \quad \boldsymbol{x}_{\mathrm{a},k+1} = \boldsymbol{F}(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k}), \quad k = 0, \ldots, N-1, \tag{47b}$$

$$s_{k+1} = s_k + T_s\, v_{s,k}, \quad k = 0, \ldots, N-1, \tag{47c}$$

$$v_{s,k+1} = v_{s,k} + T_s\, a_{s,k}, \quad k = 0, \ldots, N-1, \tag{47d}$$

$$\boldsymbol{x}_{\mathrm{a},0} = \boldsymbol{x}_{\mathrm{a,init}}, \tag{47e}$$

$$f_{\min} \le f \le f_{\max}, \tag{47f}$$

$$|\tau_i| \le \tau_{\max}, \quad i = 1, 2, 3, \tag{47g}$$

$$a_{s,\min} \le a_s \le a_{s,\max}, \tag{47h}$$

$$0 \le s \le s_{\max}, \qquad 0 \le v_s \le v_{s,\max}, \tag{47i}$$

where $\boldsymbol{F}(\boldsymbol{x}_{\mathrm{a},k}, \boldsymbol{u}_{\mathrm{a},k})$ denotes one step of the augmented dynamics (22) and (42) integrated by an implicit Runge-Kutta scheme. The bound constraint (47i) confines the virtual reference point to the valid domain of the trajectory interpolant,

preventing extrapolation beyond $s_{\max}$ while ensuring that the virtual progress speed remains dynamically consistent with the second-order progress model.

Although the DQ-MPCC formulation is derived using the nominal torque-driven quadrotor model, the Software-in-the-Loop validation described in Section VII-C employs a practical body-rate-and-thrust command interface consistent with the onboard flight stack used in the real platform.

V. Bilevel Optimisation for Weight Tuning

The stage cost in (45) depends on a vector of tuning weights, collected as

$$\boldsymbol{\vartheta} = \left[\, \boldsymbol{q}_c^\top,\ \boldsymbol{q}_\ell^\top,\ \boldsymbol{q}_q^\top,\ \boldsymbol{w}_u^\top,\ \boldsymbol{q}_\omega^\top,\ Q_s \,\right]^\top \in \mathbb{R}^{17}, \tag{48}$$

with $\boldsymbol{q}_c = [Q_c^x, Q_c^y, Q_c^z]^\top$, $\boldsymbol{q}_\ell = [Q_\ell^x, Q_\ell^y, Q_\ell^z]^\top$, $\boldsymbol{q}_q = [Q_q^x, Q_q^y, Q_q^z]^\top$, $\boldsymbol{w}_u = [U^f, U^{\tau_1}, U^{\tau_2}, U^{\tau_3}]^\top$, and $\boldsymbol{q}_\omega = [Q_\omega^x, Q_\omega^y, Q_\omega^z]^\top$. The associated diagonal weighting matrices are defined as $\boldsymbol{Q}_\ell = \mathrm{diag}(\boldsymbol{q}_\ell)$, $\boldsymbol{Q}_c = \mathrm{diag}(\boldsymbol{q}_c)$, $\boldsymbol{Q}_q = \mathrm{diag}(\boldsymbol{q}_q)$, $\boldsymbol{Q}_\omega = \mathrm{diag}(\boldsymbol{q}_\omega)$, and $\boldsymbol{W}_u = \mathrm{diag}(\boldsymbol{w}_u)$. The additional scalar Q_a appearing in (45) is kept fixed throughout the bilevel optimisation in order to isolate the effect of the main tracking, effort, and progress weights on the closed-loop behaviour.

Since these parameters are mutually coupled and may span different orders of magnitude, manual tuning alone is difficult to reproduce. In this work the weight-selection workflow is therefore split into two stages. First, a coarse manual adjustment is carried out until a stable and practically useful closed-loop behaviour is obtained, yielding a reference vector $\boldsymbol{\vartheta}_{\mathrm{ref}}$. Second, bilevel optimisation is used only as a *local* refinement around this reference point. This preserves the qualitative behaviour obtained by engineering judgement while avoiding an overly broad search domain.

During this local refinement, all entries of $\boldsymbol{\vartheta}$ except Q_a are allowed to vary inside a bounded neighbourhood of the manual reference. The outer level therefore acts on the local decision vector

$$\boldsymbol{\psi} = \left[\, \boldsymbol{q}_c^\top,\ \boldsymbol{q}_\ell^\top,\ \boldsymbol{q}_q^\top,\ \boldsymbol{w}_u^\top,\ \boldsymbol{q}_\omega^\top,\ Q_s \,\right]^\top \in \mathbb{R}^{17}, \tag{49}$$

for the Euclidean MPCC baseline. For the DQ-MPCC controller, the same local-refinement logic is used, replacing the rotational block $\boldsymbol{q}_q$ by the corresponding Lie-algebra weight vector $\boldsymbol{q}_\phi$. In both cases, the only cost coefficient excluded from the outer decision vector is Q_a.

A. Bilevel Problem Formulation

Let $\mathcal{V}$ denote a representative set of operating conditions. For each candidate vector $\boldsymbol{\psi}$ and each $v \in \mathcal{V}$, the *inner-level* problem consists of solving the OCP in Section **??** at each sampling instant and simulating the resulting closed-loop rollout over the full trajectory. From this rollout, the outer level extracts contouring and lag RMSE values together with completion, solver-failure, and path-speed metrics.

The *outer-level* optimisation problem is then written as

$$\boldsymbol{\psi}^\star = \arg\min_{\boldsymbol{\psi}\in\Theta_{\mathrm{loc}}} J_{\mathrm{multi}}(\boldsymbol{\psi}), \qquad J_{\mathrm{multi}}(\boldsymbol{\psi}) = \frac{1}{|\mathcal{V}|}\sum_{v\in\mathcal{V}} J(\boldsymbol{\psi}, v), \tag{50}$$

where Θ_{loc} denotes a local neighbourhood around the manual reference controller $\boldsymbol{\vartheta}_{\mathrm{ref}}$.

For both controllers, each sub-objective is defined as

$$\begin{aligned} J(\boldsymbol{\psi}, v) = {} & W_c\,\frac{\mathrm{RMSE}_c}{\mathrm{RMSE}_c^{\mathrm{ref}}(v)} + W_\ell\,\frac{\mathrm{RMSE}_\ell}{\mathrm{RMSE}_\ell^{\mathrm{ref}}(v)} \\ & - W_v\,\frac{v_{\mathrm{eff}}}{v_{\mathrm{eff}}^{\mathrm{ref}}(v)} + W_{\mathrm{inc}}\,[1-c]^+ + W_{\mathrm{fail}}\,\eta_{\mathrm{fail}}, \end{aligned} \tag{51}$$

where $[\cdot]^+ = \max(\cdot, 0)$, $c = \theta_K / s_{\max}$ is the path-completion ratio, η_{fail} is the mean solver-failure ratio, and

$$\bar{\rho} = \frac{1}{K}\sum_{k=0}^{K-1} \frac{v_{\mathrm{path},k}}{v_{s,k} + \varepsilon}, \qquad v_{\mathrm{eff}} = \bar{v}_s\,\bar{\rho}, \tag{52}$$

with $\bar{v}_s$ the mean virtual progress speed and $v_{\mathrm{path},k}$ the realised path-relative speed at step k. The superscript "ref" denotes the metrics obtained with the manually tuned controller $\boldsymbol{\vartheta}_{\mathrm{ref}}$ at the same operating condition v.

These upper-level costs are aligned with the practical tuning goal of the paper: reduce contouring and lag while preserving a physically meaningful relation between virtual progress and actual spatial motion. The term based on v_{eff} rewards fast traversal only when the virtual progress remains consistent with the realised path speed, thereby discouraging solutions that improve the arc-length state artificially without improving the true path-following behaviour.

The outer-level optimisation is solved with Bayesian optimisation, while the inner level is the full closed-loop controller evaluation.

VI. Experimental Setup

Numerical simulations are carried out on a desktop workstation equipped with an AMD Ryzen 7 7000 series processor running Ubuntu 22.04, while the experimental validation is conducted on a custom quadrotor platform with an NVIDIA Jetson Orin NX 16 GB as onboard computer.

The high-level control loop runs at $100\,\mathrm{Hz}$, and the solver consistently executes below this timing bound for a prediction horizon of $0.3\,\mathrm{s}$. The second-order progress model is handled inside the same OCP, so no additional smoothing layer is required outside the solver. A more detailed analysis of the effect of the prediction horizon and the number of discretization steps N is left for future work. This setup enables evaluation of the full closed-loop software stack under flight-representative computational conditions while preserving consistency with the command pathway used in the real platform. The corresponding simulator approximation is described next.

The OCP (47) is solved within the ACADOS framework [12], employing a Sequential Quadratic Programming Real-Time Iteration (SQP-RTI) scheme that executes a single QP per control cycle [18]. Quadratic subproblems are handled by HPIPM with full condensing, while system dynamics are integrated via an Implicit Runge-Kutta (IRK) scheme. The Hessian is approximated via Gauss-Newton with CONVEXIFY regularization, and stage costs are expressed as CasADi symbolic externals. The controller operates at a sample time of $T_s = 10\,\mathrm{ms}$ over a prediction horizon of $T_N = 0.3\,\mathrm{s}$, discretised into $N = 30$ shooting nodes.

A critical implementation aspect is the representation of the reference trajectory $\boldsymbol{\gamma}(s)$ as a CasADi symbolic function compatible with automatic differentiation in ACADOS. The map is constructed from $N_w = 30$ arc-length waypoints over $s \in [0, s_{\max}]$ using piecewise-linear interpolation of the dual-quaternion and velocity components. Quaternion continuity is preserved by enforcing hemisphere consistency, i.e., flipping $q^d_{r,i} \leftarrow -q^d_{r,i}$ whenever $q^d_{r,i-1} \cdot q^d_{r,i} < 0$, while the heading angle is unwrapped before interpolation.

For the experiments in this work, the bilevel tuning procedure is instantiated using the representative velocity set

$$\mathcal{V} = \{5,\ 10,\ 16\}\ \text{m/s}.$$

Each tuning rollout uses the same headless MiL configuration: maximum simulated duration $T_{\text{final}} = 30\,\text{s}$, trajectory-definition window $T_{\text{traj}} = 30\,\text{s}$, control frequency $f_s = 100\,\text{Hz}$ ($T_s = 10\,\text{ms}$), and prediction horizon $T_N = 0.3\,\text{s}$ discretised into $N = 30$ shooting nodes with $N_w = 30$ arc-length interpolation waypoints. Under this setup, one rollout contains approximately 2971 closed-loop simulation steps, and each Optuna trial evaluates one full rollout for every operating point in $\mathcal{V}$.

After coarse manual tuning, the local search domain is defined around the reference controller $\boldsymbol{\vartheta}_{\text{ref}}$ using hierarchical multiplicative windows. The tracking and orientation blocks $\boldsymbol{q}_c$, $\boldsymbol{q}_\ell$, and $\boldsymbol{q}_q$ or $\boldsymbol{q}_\phi$ are searched within $[0.75,\ 1.35] \times (\cdot)^{\text{ref}}$, the progress weight is searched within $[0.75,\ 1.35] \times Q_s^{\text{ref}}$, the effort block $\boldsymbol{w}_u$ is searched within $[0.92,\ 1.08] \times (\boldsymbol{w}_u)^{\text{ref}}$, and the angular-rate block $\boldsymbol{q}_\omega$ is searched within $[0.90,\ 1.10] \times (\boldsymbol{q}_\omega)^{\text{ref}}$. The progress-acceleration weight Q_a remains fixed at its manually tuned value. This choice avoids confounding the comparison of tracking-oriented cost weights with the internal smoothing imposed on the second-order virtual-progress dynamics, so the bilevel refinement focuses on the blocks that directly shape path-following behaviour. The resulting local search space is reported in Table I.

The same upper-level coefficients are used for both controllers, so that the outer bilevel comparison remains methodologically consistent across DQ-MPCC and the Euclidean MPCC baseline. In the present study, the local objective in (51) is instantiated with $W_c = 1.5,\ W_\ell = 0.35,\ W_v = 0.80,\ W_{\text{inc}} = 1000,\ W_{\text{fail}} = 500$.

The outer optimisation is performed with the Tree-structured Parzen Estimator (TPE) algorithm implemented in Optuna [20], using 10 uniform warm-up trials and random seed 42 for reproducibility. Since each trial requires one full closed-loop simulation for each operating point in $\mathcal{V}$, the computational cost of each outer iteration is approximately $|\mathcal{V}|$ times that of a single simulation run. In the current study, the final local refinement stage is carried out with 100 TPE trials per controller, which provides sufficient exploration of the bounded neighbourhood around the manually tuned operating point without resorting to a full high-dimensional global search.

TABLE I: Local search window Θ_{loc} used in the bilevel refinement around the manually tuned reference controller ($i \in \{x, y, z\}$).

Parameter	Local range
$Q_{c,i}$	$[0.75,\ 1.35] \times Q^{\text{ref}}_{c,i}$
$Q_{\ell,i}$	$[0.75,\ 1.35] \times Q^{\text{ref}}_{\ell,i}$
$Q_{q,i}$ or $Q_{\phi,i}$	$[0.75,\ 1.35] \times Q^{\text{ref}}_{q,i}$ or $Q^{\text{ref}}_{\phi,i}$
Q_s	$[0.75,\ 1.35] \times Q^{\text{ref}}_s$
$U^f,\ U^{\tau_j}$	$[0.92,\ 1.08] \times (U^f,\ U^{\tau_j})^{\text{ref}}$
$Q_{\omega,i}$	$[0.90,\ 1.10] \times Q^{\text{ref}}_{\omega,i}$
Q_a	fixed at manual reference

The reference path is a three-dimensional Lissajous curve, defined in closed form as

$$\boldsymbol{p}^d(t) = \begin{bmatrix} A_x \sin(\Omega_x t + \delta_x) \\ A_y \sin(\Omega_y t) \\ A_z \cos(\Omega_z t) + z_0 \end{bmatrix}, \tag{53}$$

where A_x, A_y, A_z are the trajectory amplitudes, $\Omega_x, \Omega_y, \Omega_z$ the path frequencies, δ_x a phase offset that breaks planar symmetry, and z_0 the nominal flight altitude. The curve is periodic with period T_L and combines low- and high-curvature segments that exercise the lag–contouring decomposition across a representative range of flight conditions. The time-parametrized path $\boldsymbol{p}^d(t)$ is evaluated analytically, so derivatives $\dot{\boldsymbol{p}}^d$ and $\ddot{\boldsymbol{p}}^d$ are available in closed form without numerical differentiation. The specific parameter values used in all experiments are reported in Table II.

All simulations are conducted on the three-dimensional Lissajous path defined in Section III-D, equation (53), with the parameters reported in Table II. The resulting curve spans a total arc length of approximately $80\,\text{m}$ over a period of $T_L = 15\,\text{s}$, combining low-curvature straight segments with tight turns that stress the lag–contouring decomposition across both controllers.

TABLE II: Lissajous Trajectory Parameters

Parameter	Value
Amplitudes ($A_x,\ A_y,\ A_z$)	(10.0, 8.0, 3.0) m
Frequencies ($\Omega_x,\ \Omega_y,\ \Omega_z$)	(0.42, 0.84, 1.26) $\text{rad}\,\text{s}^{-1}$
Phase offset δ_x	$\pi/2$ rad
Nominal altitude z_0	5.0 m
Period T_L	15.0 s
Arc length $s_{\max}$	≈ 80 m

VII. Results

A. MiL Simulations

1) Local Tuning Report: The local refinement reported here follows the common bilevel formulation and experimental setup defined in Section V-A and Section VI. Table III reports the manual and refined gains selected by the local search, while Table IV summarises the corresponding closed-loop effect in the final MiL baseline runs.

TABLE III: Local tuning summary for both controllers.

Weight	Manual	DQ-MPCC	MPCC
$\boldsymbol{q}_\phi/\boldsymbol{q}_q$	$[0.65, 0.65, 1.9]$	$[0.53, 0.69, 1.7]$	$[0.62, 0.72, 1.6]$
$\boldsymbol{q}_c$	$[40.0, 40.0, 40.0]$	$[44.4, 37.9, 32.6]$	$[40.6, 35.9, 33.7]$
$\boldsymbol{q}_\ell$	$[3.0, 3.0, 3.0]$	$[4.0, 2.6, 3.6]$	$[4.0, 4.0, 2.7]$
$\boldsymbol{w}_u$	$[0.050, 70.0, 70.0, 70.0]$	$[0.049, 68.9, 69.6, 66.6]$	$[0.049, 70.9, 67.6, 72.2]$
$\boldsymbol{q}_\omega$	$[0.020, 0.020, 0.10]$	$[0.021, 0.021, 0.100]$	$[0.020, 0.021, 0.095]$
Q_s	11.0	8.4	8.3
$J^\star$	–	0.917	0.863
Trial	–	#72	#94

TABLE IV: Closed-loop comparison between manual and refined gains used in the final baseline runs.

Controller	Variant	$\mathbf{RMSE}_c$	$\mathbf{RMSE}_\ell$	$\mathbf{v}_{\text{eff}}$	Path	Fail
DQ-MPCC	Manual	0.346	0.507	1.092	100.00%	0.00%
DQ-MPCC	Refined	0.291	0.335	1.216	100.00%	0.00%
DQ-MPCC	Δ (%)	16.1	33.9	11.3	–	–
MPCC	Manual	0.445	0.974	5.134	99.98%	0.00%
MPCC	Refined	0.337	0.476	3.979	99.98%	0.00%
MPCC	Δ (%)	24.4	51.1	-22.5	–	–

Overall, both controllers benefited from the local refinement. While the resulting closed-loop effects differ between controllers, this stage is important to ensure that the subsequent comparison is conducted on a fairer and more balanced basis, since both controllers are evaluated after the same consistent local tuning procedure.

2) Velocity Sweep: Contouring and Lag Error Analysis: To rigorously evaluate the tracking quality of both formulations across operating regimes, we decompose the position tracking error into its orthogonal components: the *contouring error* e_c (perpendicular distance to the reference path) and the *lag error* e_ℓ (signed projection along the path tangent). By the Pythagorean relationship these satisfy

$$\mathrm{RMSE}_{e_p}^2 = \mathrm{RMSE}_{e_c}^2 + \mathrm{RMSE}_{e_\ell}^2, \tag{54}$$

which was verified numerically for every run in our Monte Carlo campaign. Both components are computed in the inertial frame to ensure a fair, frame-neutral comparison between the DQ-MPCC (which internally operates in the body frame via $\mathrm{Log}\big(\mathrm{SE}(3)\big)$) and the Baseline MPCC (inertial-frame errors).

The experiment considers eight maximum progress velocities,

$$v_\theta^{\max} \in \{4,\ 6,\ 8,\ 10,\ 12,\ 15,\ 16,\ 18\}\ \text{m/s},$$

with 10 Monte Carlo runs per velocity and per controller ($\sigma_p = 0.05$ m, $\sigma_q = 0.05$ rad), for a total of 160 simulations along a Lissajous path of arc length $s_{\max} = 80$ m.

TABLE VI: Orthogonal decomposition of the position error into contouring ($\%e_c^2$) and lag ($\%e_\ell^2$) components. The ratio e_ℓ/e_c indicates the dominant error type: > 1 implies lag-dominated tracking, < 1 implies contouring-dominated.

	DQ-MPCC			Baseline MPCC		
$v_\theta^{\max}$	$\%e_c^2$	$\%e_\ell^2$	e_ℓ/e_c	$\%e_c^2$	$\%e_\ell^2$	e_ℓ/e_c
4	53.0	47.0	0.942	69.7	30.3	0.660
6	49.4	50.6	1.011	66.7	33.3	0.706
8	47.6	52.4	1.049	68.6	31.4	0.678
10	46.0	54.0	1.084	62.6	37.4	0.773
12	44.7	55.3	1.113	60.2	39.8	0.813
15	40.8	59.2	1.206	59.1	40.9	0.833
16	39.7	60.3	1.234	60.9	39.1	0.801
18	37.6	62.4	1.287	61.0	39.0	0.799

Fig/fig_pareto_quad.pdf

Fig. 2: Speed–accuracy Pareto frontiers for the DQ-MPCC and Baseline MPCC across 8 progress velocities. Each point is the median over 10 Monte Carlo runs; error bars denote the interquartile range (P25–P75). (a) Contouring RMSE, (b) Lag RMSE, (c) Position RMSE, (d) Orientation RMSE, all vs. lap time. Annotations indicate $v_\theta^{\max}$ [m/s].

Fig. 2 presents the speed–accuracy Pareto frontiers for both controllers. In panel (a), the DQ-MPCC (blue) lies below the Baseline (red) for $v_\theta^{\max} < 15$ m/s, after which the curves nearly merge (gap < 0.010 m at 18 m/s), confirming that the coupled $\mathrm{SE}(3)$ formulation yields tighter geometric adherence to the path. Panel (b) reveals the opposite: the Baseline's lag curve is consistently lower, with an absolute gap that widens from 0.053 m at 4 m/s to 0.277 m at 18 m/s, reflecting the decoupled formulation's ability to independently minimise along-path delay. Panel (c) combines both components via (54): the curves cross near $v_\theta^{\max} \approx 8$ m/s, with the lag-dominated DQ-MPCC yielding higher total position error at high speeds. Finally, panel (d) shows that the DQ-MPCC maintains consistently lower orientation error across the entire

TABLE V: Comparative performance across progress velocities. $\Delta\%$ is computed as $(x_{\text{DQ}} - x_{\text{Base}})/x_{\text{Base}} \times 100$; negative values favour DQ-MPCC. Bold marks the better value per velocity.

$v_\theta^{\max}$	Contouring RMSE_{e_c} [m]			Lag RMSE_{e_ℓ} [m]			Position RMSE_{e_p} [m]			Orientation RMSE_{e_ψ} [rad]			Lap time t_{lap} [s]		
	DQ	Base	$\Delta\%$	DQ	Base	$\Delta\%$	DQ	Base	$\Delta\%$	DQ	Base	$\Delta\%$	DQ	Base	$\Delta\%$
4	**0.2863**	0.3287	-12.9%	0.2698	**0.2168**	+24.4%	**0.3934**	0.3938	-0.1%	**0.3808**	0.4972	-23.4%	22.29	24.03	-7.3%
6	**0.3294**	0.3850	-14.4%	0.3332	**0.2719**	+22.5%	**0.4685**	0.4713	-0.6%	**0.5338**	0.6767	-21.1%	15.60	16.47	-5.3%
8	**0.3869**	0.4549	-14.9%	0.4059	**0.3083**	+31.6%	0.5608	**0.5498**	+2.0%	**0.6561**	0.8330	-21.2%	12.24	12.71	-3.8%
10	**0.4340**	0.4521	-4.0%	0.4705	**0.3495**	+34.6%	0.6401	**0.5715**	+12.0%	**0.7520**	0.8956	-16.0%	10.18	10.10	+0.8%
12	**0.4793**	0.4800	-0.1%	0.5335	**0.3904**	+36.6%	0.7172	**0.6188**	+15.9%	**0.8239**	0.9995	-17.6%	8.74	8.81	-0.7%
15	0.5312	**0.5118**	+3.8%	0.6404	**0.4262**	+50.3%	0.8321	**0.6660**	+24.9%	**0.9225**	1.0996	-16.1%	7.37	7.42	-0.6%
16	0.5482	**0.5479**	+0.1%	0.6764	**0.4391**	+54.0%	0.8707	**0.7021**	+24.0%	**0.9490**	1.1123	-14.7%	7.05	7.05	-0.0%
18	**0.5828**	0.5929	-1.7%	0.7504	**0.4738**	+58.4%	0.9502	**0.7590**	+25.2%	**0.9902**	1.1462	-13.6%	6.51	6.49	+0.4%
Avg.	0.4473	0.4692	-4.7%	0.5100	0.3595	+41.9%	0.6791	0.5915	+14.8%	0.7510	0.9075	-17.2%	11.25	11.63	-3.3%

velocity range (13.6–23.4%), a direct benefit of the pose-level coupling in the cost function.

Fig/fig_3d_velocity.pdf

Fig. 3: Three-dimensional closed-loop trajectories for both controllers across multiple progress-velocity limits. Grey curves correspond to $v_\theta^{\max} \in \{4, 6, 8, 12, 14, 16, 18, 20\}$ m/s, while the red curve highlights $v_\theta^{\max} = 10$ m/s as the nominal operating point. The dashed black curve is the arc-length-parameterised reference path ($s_{\max} = 80$ m). The DQ-MPCC maintains tighter spatial adherence in high-curvature regions, whereas the baseline tends to achieve smaller lag at the expense of larger orientation and contouring deviations.

3) Contouring vs. Lag Decomposition: Table V shows that the DQ-MPCC achieves lower contouring error for $v_\theta^{\max} < 15$ m/s (avg. -4.7%, best -14.9% at 8 m/s), whereas the Baseline MPCC dominates in lag error (avg. $+41.9\%$, up to $+58.4\%$ at 18 m/s). The decomposition in Table VI explains this asymmetry: the Baseline is *contouring-dominated* (59–70% of $\|e_p\|^2$ comes from e_c, $e_\ell/e_c < 1$ at all speeds), while the DQ-MPCC transitions from near parity (53/ 47% at 4 m/s) to *lag-dominated* (38/ 62%, $e_\ell/e_c = 1.29$ at 18 m/s). The root cause is structural: the DQ-MPCC's $\text{SE}(3)$ logarithmic error map $\boldsymbol{\rho} = \mathbf{J}_l(\boldsymbol{\varphi})^{-1}\mathbf{R}_d^\top \Delta\mathbf{p}$ couples position and orientation, preventing the solver from independently minimising the tangential projection. The Baseline, by contrast, penalises e_c and e_ℓ as decoupled cost terms, enabling aggressive lag reduction at the expense of orientation fidelity. These results therefore do not support a blanket claim of superiority of one controller over the other; rather, they show that the proposed DQ-MPCC systematically shifts the closed-loop trade-off toward better contouring and attitude consistency, while the Euclidean baseline remains more effective at reducing tangential lag.

B. Experiment 3: Monte Carlo Robustness Evaluation

To evaluate robustness under uncertain initial conditions, we perform a Monte Carlo study with $N = 100$ trials at a fixed progress-velocity limit $v_\theta^{\max} = 10$ m/s. Each trial samples a random initial pose: position is perturbed as $\mathbf{p}_0 \sim \mathcal{N}(\bar{\mathbf{p}}_0, \sigma_p^2 \mathbf{I}_3)$ with $\sigma_p = 2.0$ m, and orientation as a random rotation with $\|\text{Log}(\delta q)\| \sim U(0, 0.5)$ rad ($\approx 29°$). The path length is $s_{\max} = 80$ m with a time budget of $T = 40$ s. A trial is deemed *successful* if the progress variable reaches $\theta \geq 95\%$ of $s_{\max}$.

TABLE VII: Monte Carlo robustness comparison ($v = 10$ m/s, $N = 100$).

Metric	**DQ-MPCC**	**Baseline**	**Δ**
Conv. rate [%]	99.0	100.0	-1.0%
RMSE e_c	0.8243 ± 0.5405	0.7632 ± 0.6192	+8.0%
RMSE e_l	0.6425 ± 0.3936	0.4827 ± 0.2650	+33.1%
RMSE e_{pos}	1.0824 ± 0.6066	0.9390 ± 0.6224	+15.3%
RMSE e_{ori}	0.9118 ± 0.1147	1.0132 ± 0.0419	-10.0%
Solver time [ms]	3.81 ± 0.11	2.04 ± 0.09	+86.7%
Ctrl. effort	33518.1 ± 75505.4	35187.1 ± 30591.3	-4.7%
Lap time [s]	10.57 ± 0.29	10.39 ± 0.30	+1.7%

a) Convergence: The baseline achieves a convergence rate of 100.0%, compared to 99.0% for DQ-MPCC. Both controllers achieve near-perfect convergence despite large initial-pose perturbations ($\sigma_p = 2.0$ m, $\sigma_q = 0.5$ rad), confirming reliable operation under uncertainty.

Fig/fig_mc_robustness.pdf

Fig. 4: Distribution of per-run RMSE across 100 Monte Carlo trials with random initial poses ($\sigma_p = 2.0$ m, $\sigma_q = 0.5$ rad) at $v_\theta^{\max} = 10$ m/s. Violin contours show probability density; individual points overlay each trial. Median values are annotated.

b) Tracking accuracy and Pareto consistency: Fig. 4 displays the error distributions for all four RMSE metrics. The orientation subplot confirms that DQ-MPCC consistently reduces e_{ori}: its median (0.8708) lies well below the baseline median (1.0046), with DQ-MPCC winning in 85 / 99 individual trials ($\Delta\bar{e}_{\text{ori}} = -10.0\%$). Crucially, the baseline's e_{ori} distribution is tightly concentrated ($\sigma = 0.0419$), while DQ-MPCC's wider spread ($\sigma = 0.1147$) reflects a pose-dependent adaptation enabled by the Lie-algebraic coupling between translation and rotation. This trade-off mirrors the Pareto front in Experiment 2 (Fig. 2): at $v = 10$ m/s with nominal initial conditions, DQ-MPCC achieves $e_c = 0.434$ (-4.0%) and $e_{\text{ori}} = 0.752$ (-16.0%) relative to the baseline. Under large pose perturbations ($\sigma_p = 2.0$ m), the transient convergence phase inflates the contouring error to $\bar{e}_c = 0.8243$ (+8.0%), yet the orientation advantage is preserved (-10.0%). Notably, the lag error e_l shows a consistent trend across both experiments (Exp. 2: $+34.6\%$, Exp. 3: +33.1%), indicating that DQ-MPCC systematically trades progress speed for orientation alignment. This trade-off is desirable for applications in which rigid-body pose consistency and attitude quality are more critical than purely tangential progress tracking.

c) Computational cost: The mean solver time per MPC step is 3.81 ms for DQ-MPCC and 2.04 ms for the baseline (+86.7%). Both remain well within the 10 ms real-time budget ($f = 100$ Hz).

d) Control effort: The accumulated control-rate effort $\sum \|\Delta\mathbf{u}\|^2$ averages 33518.1 for DQ-MPCC and 35187.1 for the baseline (-4.7%), indicating smoother actuation for the dual-quaternion formulation.

C. Simulation Experiment

Prior to flight tests, both controllers are validated in a MuJoCo-based Software-in-the-Loop (SiL) setup executed on the Jetson Orin NX. Because the onboard flight stack accepts body-rate and thrust commands rather than direct torque inputs, the simulator is configured to reproduce the same control interface.

Unlike the nominal rigid-body model introduced in Section III, where the rotational dynamics are expressed in terms of applied control torques, the MuJoCo simulator employs a closed-loop approximation of the onboard angular-rate controller. Specifically, the rotational subsystem is represented by the first-order response

$$\dot{\boldsymbol{\omega}} = -\mathbf{K}_\omega \left(\boldsymbol{\omega} - \boldsymbol{\omega}_{\text{cmd}}\right), \tag{55}$$

where $\boldsymbol{\omega} \in \mathbb{R}^3$ is the measured body angular velocity, $\boldsymbol{\omega}_{\text{cmd}} \in \mathbb{R}^3$ is the commanded body-rate vector, and $\mathbf{K}_\omega = \text{diag}(k_x, k_y, k_z)$ is a diagonal gain matrix that approximates the effective bandwidth of the inner rate loop.

Accordingly, the simulator input is defined as

$$\boldsymbol{u}_{\text{sil}} = \begin{bmatrix} f & \omega_{x,\text{cmd}} & \omega_{y,\text{cmd}} & \omega_{z,\text{cmd}} \end{bmatrix}^\top, \tag{56}$$

which matches the practical command interface used by the flight stack.

This SiL approximation is introduced only for implementation-level validation and does not replace the nominal control-oriented model used in the DQ-MPCC derivation. Rather, it serves as a closed-loop surrogate of the onboard rate controller, enabling a more realistic assessment of timing, numerical behaviour, and command consistency prior to real-world flight tests.

D. Real Experiment

References

[1] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.

[2] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3340–3356, 2022.

[3] J. Kabzan *et al.*, "AMZ Driverless: The full autonomous racing system," *Journal of Field Robotics*, vol. 37, no. 7, pp. 1267–1294, 2020.

[4] X. Wang and C. Yu, "Unit dual quaternion-based feedback linearization tracking problem for attitude and position dynamics," *Systems & Control Letters*, vol. 61, no. 12, pp. 1261–1269, 2012.

[5] H. Abaunza, L. Montes, and C. Ferreira, "Dual quaternion modeling and control of a quad-rotor aerial manipulator," *IEEE Latin America Transactions*, vol. 15, no. 2, pp. 220–228, 2017.

[6] B. Kenwright, "A beginners guide to dual-quaternions," in *Proc. 20th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision*, 2012, pp. 1–13.

[7] T. A. Johansen, T. I. Fossen, S. M. A. Shah, and A. S. de Oliveira, "Dual quaternion control: A review of recent results within motion control," *Nonlinear Studies*, vol. 26, no. 2, pp. 265–281, 2019.

[8] K. Daniilidis, "Hand-eye calibration using dual quaternions," *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999.

[9] J. M. Selig, *Geometric Fundamentals of Robotics*, 2nd ed. Springer, 2005.

[10] J. Solà, J. Deray, and D. Atchuthan, "A micro Lie theory for state estimation in robotics," *arXiv preprint arXiv:1812.01537*, 2018.

[11] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
[12] R. Verschueren *et al.*, "acados—A modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, pp. 147–183, 2022.
[13] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
[14] F. C. Park, J. E. Bobrow, and S. R. Ploen, "A Lie group formulation of robot dynamics," *The International Journal of Robotics Research*, vol. 14, no. 6, pp. 609–618, 1995.
[15] F. Bullo and R. M. Murray, "Proportional derivative (PD) control on the Euclidean group," in *Proc. European Control Conference*, 1995, pp. 1091–1097.
[16] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proc. IEEE Conference on Decision and Control*, 2010, pp. 5420–5425.
[17] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.
[18] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM Journal on Control and Optimization*, vol. 43, no. 5, pp. 1714–1736, 2005.
[19] J. Baumgarte, "Stabilization of constraints and integrals of motion in dynamical systems," *Computer Methods in Applied Mechanics and Engineering*, vol. 1, no. 1, pp. 1–16, 1972.
[20] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.